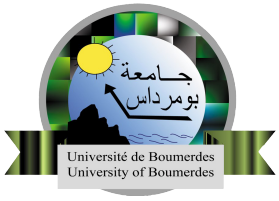
# FPGA-Accelerated Network IDS with RISC-V Co-Processor Integration




Youcef Slimani
Salah Eddine Laifa

Institute of Electrical and Electronic Engineering

Fundamental teaching department

M'Hamed Bougara University of Boumerdes

Submitted in partial satisfaction of the requirements for the
Degree of Bachelor
in Electrical and Electronic Engineering

*Supervisor* Dr.Touzzout Walid

May 2026

# Dedications

This thesis is dedicated to our lovely parents for their endless support and encouragement and to our dear brothers. We further extend our dedication to all members of the family.

We dedicate this work also without exception to all our friends, colleagues, and teachers.

Last and not least, I dedicate our work to everyone who has taught, encouraged, and advised us during all our studies.

# Acknowledgments

We would like to express my deepest gratitude to our supervisor, Dr.Touzzout Walid, for his invaluable guidance, patience, and constant encouragement throughout this project. Their expertise and insights were instrumental in the completion of this work.

I would also like to thank the IGEE(ex.inelec) Crew for providing the necessary facilities and resources to conduct this project.

Finally, We are forever grateful to our families, especially our parents, for their unwavering love and belief in us during this long journey.

# Abstract

This project presents an integrated hardware-software system combining a pipelined RISC-V processor with an FPGA-accelerated network intrusion detection system (IDS). The work is divided into two complementary components: a 32-bit RISC-V processor (RF32ICore) implementing the RV32I Base Integer ISA, and a multilayered hardware IDS running on the programmable logic of a Zynq-7020 SoC. The RISC-V processor was developed using VHDL and verified through simulation with C programs of increasing complexity, demonstrating correct execution of the full RV32I instruction set. The IDS implements a layered pipeline architecture processing network packets in real time, detecting 26 attack patterns across Ethernet, IPv4, TCP, and UDP protocols. The system employs a novel co-design approach where the RISC-V processor adapts IDS rules at runtime through a memory-mapped register interface, enabling dynamic threat scoring and adaptive sensitivity tuning. Integration is achieved via AXI-Stream DMA between the Zynq Processing System (which receives Ethernet packets) and the Programmable Logic fabric (which executes the IDS pipeline). Both components are fully verified through simulation, and the combined system demonstrates the feasibility of custom RISC-V implementations integrated with hardware-accelerated security processing for next-generation embedded systems.

# Table of Contents

# List of Figures

# List of Tables

# Abbreviations

| | |
|---|---|
| ALU | Complex Instruction Set Computer |
| BRAM | Block RAM |
| CISC | Complex Instruction Set Computer |
| DMA | Direct Memory Access |
| FCS | Frame Check Sequence |
| GCC | GNU Compiler Collection |
| NOP | No Operation |
| GigE | Gigabit Ethernet |
| HDL | Hardware Description Language |
| IDS | Intrusion Detection System |
| IHL | Internet Header Length |
| ISA | Instruction Set Architecture |
| LUT | Look-Up Table |
| MAC | Media Access Control |
| MOSFET | Metal-Oxide-Semiconductor Field-Effect Transistor |
| MREQ | Memory Request |
| PCB | Printed Circuit Board |
| PCI | Physical Cell ID |
| PHY | Physical Interface |
| PL | Programmable Logic |
| PS | Processing System |
| RAM | Random Access Memory |
| RISC | Reduced Instruction Set Computer |
| RMII | Reduced Media-Independent Interface |
| RTL | Register Transfer Level |
| RV32I | RISC-V 32-bit Base Integer ISA |
| SoC | System-on-Chip |
| TCP | Transmission Control Protocol |
| VHDL | VHSIC Hardware Description Language |
| XMAS | Christmas Scan |

# Chapter 1
# Introduction

Network traffic volumes and attack sophistication have grown to a point where software intrusion detection systems face fundamental throughput limitations. A software IDS running on a general-purpose CPU must copy packet data through the OS network stack, schedule the inspection process, and share the CPU with other tasks—all of which introduce variable latency and reduce the fraction of packets that can actually be inspected at high line rates.

Reconfigurable hardware offers a different trade-off: inspection logic is implemented directly as digital circuits in an FPGA, so packets are examined as bytes arrive, with no operating system, no scheduling overhead, and no data copies. The cost is development complexity; the benefit is deterministic, wire-speed inspection.

This project builds a hardware IDS in VHDL, targeting the MicroPhase Z7-Lite development board (Zynq-7020, XC7Z020-1CLG400C), and integrates it with a RISC-V processor built independently by a collaborator. The Zynq device is a natural fit: its ARM Cortex-A9 Processing System (PS) receives Ethernet packets through the on-chip GigE MAC and can forward raw frame data to the Programmable Logic (PL) fabric via AXI-Stream DMA, where the IDS pipeline runs. The RISC-V processor also resides in the PL fabric and communicates with the IDS through a shared memory-mapped register bank.

The IDS is developed and validated entirely through behavioural simulation before any hardware integration, allowing each layer of the protocol stack to be verified independently with injected test packets.

# Chapter 2

# RISC-V Theoretical Background & System Architecture

## 2.1 Introduction

This chapter presents the theoretical foundations of RV32ICore, covering the RISC-V ISA, core processor design concepts, and the system architecture of the implemented pipeline.

## 2.2 RISC-V Instruction Set Architecture

### 2.2.1 History and Origin

RISC-V was conceived in 2010 at UC Berkeley by Krste Asanović, Andrew Waterman, and Yunsup Lee under David Patterson. It was designed to be open, stable, and free from proprietary constraints. The RISC-V Foundation was established in 2015 and transferred to RISC-V International in 2020 to ensure long-term neutrality.

### 2.2.2 Design Philosophy

RISC-V adheres to RISC principles: simplicity, regularity, and extensibility. The base ISA is minimal, with optional standard extensions for floating-point, atomics, compressed instructions, and vector processing. Instructions are fixed 32-bit wide, and frequently used fields (source/destination registers) appear at fixed bit positions across all formats, reducing decoder complexity.

### 2.2.3 RV32I Instruction Formats

RV32I defines six fixed-width 32-bit instruction formats:

- **R-type** — register-to-register operations; encodes rs1, rs2, rd, funct3, funct7.

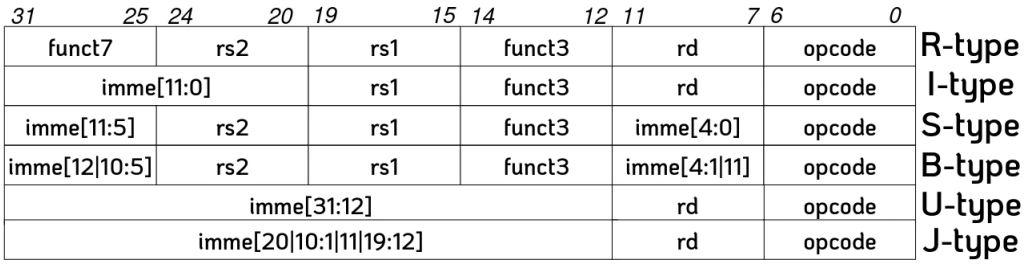



Figure 2.1: RV32I Instruction Formats

- **I-type** — immediate arithmetic, loads, and environment calls; encodes rs1, rd, 12-bit signed immediate.
- **S-type** — store instructions; 12-bit immediate split across two fields to keep register fields fixed.
- **B-type** — conditional branches; 13-bit PC-relative offset with shuffled immediate bits.
- **U-type** — upper immediate instructions (LUI, AUIPC); 20-bit immediate and rd.
- **J-type** — JAL only; 21-bit PC-relative offset and rd for the return address.

## 2.3 Processor Design Concepts

### 2.3.1 Von Neumann vs. Harvard Architecture

Von Neumann architecture uses a single shared bus for instructions and data, introducing a fetch/access bottleneck. Harvard architecture uses separate memories and buses, allowing simultaneous instruction fetch and data access. A Modified Harvard design — separate L1 caches with unified main memory — is the practical compromise used in most modern processors.

RV32ICore adopts a Harvard architecture with fully separate instruction and data memories, ensuring that instruction fetch and data memory access never contend for the same resource.

### 2.3.2 Single-Cycle vs. Pipelined Execution

A single-cycle processor completes one instruction per clock cycle but requires the clock period to accommodate the slowest instruction, limiting frequency. A pipelined processor divides the datapath into stages, overlapping instruction execution to approach one instruction per cycle throughput, with the clock period set by the slow-

est stage. This introduces structural, data, and control hazards, managed through stalling, forwarding, and flushing.

RV32ICore implements a five-stage pipeline (IF, ID, EX, MEM, WB) with forwarding and stall-based hazard resolution and flush-based control hazard handling.

## 2.4 Top-Level Architecture

RV32ICore is composed of three units:

**Datapath** — processes and routes data through the pipeline. Inputs: control signals, instructions, and data memory output. Outputs: PC to instruction memory, write data and address to data memory, and status signals to the Control Unit.

**Control Unit** — decodes opcode[6:0], Funct3[14:12], and Funct7[30] to generate control signals governing multiplexers, the register file, ALU, and data memory.

**Hazard Unit** — monitors pipeline registers and generates stall and flush signals to maintain correctness under data dependencies and control flow changes.

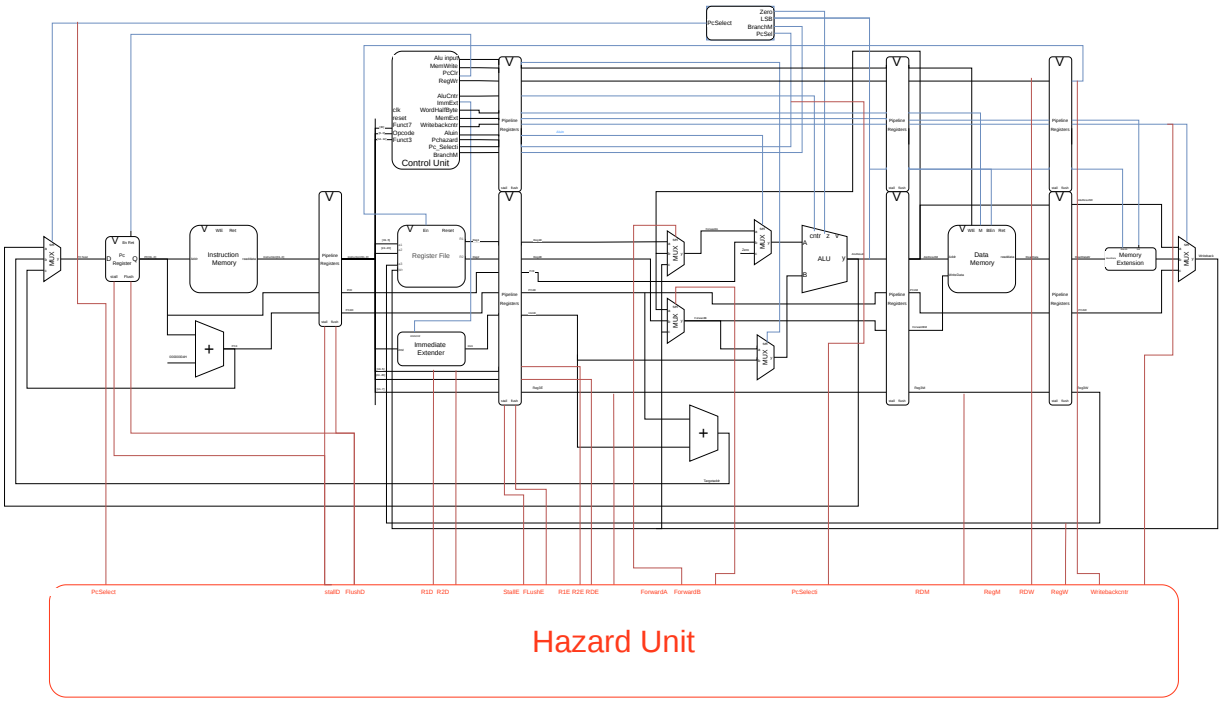



Figure 2.2: RV32ICore Top-Level Architecture

## 2.5 Pipeline Stages

RV32ICore implements a classical five-stage pipeline. Each stage is separated by a pipeline register with synchronous reset and NOP-insertion control signals.

### 2.5.1 Instruction Fetch (IF)

The PC register updates on every rising clock edge. Its value addresses instruction memory, which returns the 32-bit instruction combinationally. A dedicated adder computes PC+4 for sequential advancement and as the return address for jump instructions. On a taken branch, the PC is updated to the branch target computed in EX in the same cycle the flush is issued.

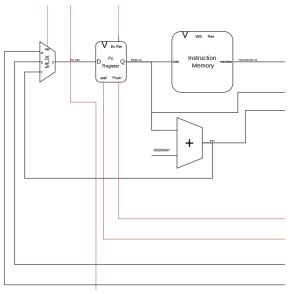



Figure 2.3: RV32ICore IF Stage

### 2.5.2 Instruction Decode (ID)

The instruction word is partitioned per the RV32I spec. rs1[19:15], rs2[24:20], and rd[11:7] address the register file combinationally; x0 is hardwired to zero. An Immediate Extension unit produces a 32-bit sign-extended immediate by selecting the appropriate format (I, S, B, U, J) from the opcode. The opcode, Funct3, and Funct7[30] are forwarded to the Control Unit, whose output control signals propagate through the pipeline with the instruction.

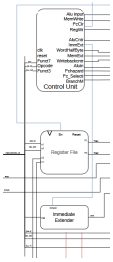



Figure 2.4: RV32ICore ID Stage

### 2.5.3 Execute (EX)

The EX stage contains the ALU, input multiplexers, branch adder, and forwarding logic.

**MuxX3** selects ALU input A from: rs1, the current PC (AUIPC), or zero (LUI). **MuxX2** selects ALU input B from: rs2 or the sign-extended immediate. The ALU performs addition, subtraction, bitwise AND/OR/XOR, logical/arithmetic shift, and signed/unsigned comparison, producing a 32-bit result and a zero flag for branch evaluation. A dedicated adder computes the branch target as PC + sign-extended immediate.

On a taken branch, the Hazard Unit flushes the IF/ID and ID/EX registers in the same cycle the PC is updated, discarding the two incorrectly fetched instructions. No branch prediction is used.

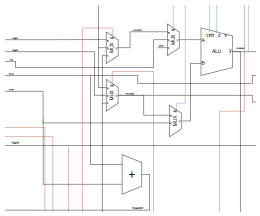


Figure 2.5: RV32ICore EX Stage

### 2.5.4 Memory Access (MEM)

For stores, the ALU result is the write address and rs2 (pipeline-carried) is the write data. For loads, the ALU result is the read address. Access width (byte, halfword, word) is controlled by Funct3. Non-memory instructions pass the ALU result through to WB without accessing data memory.

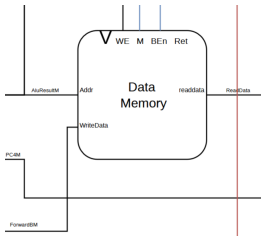



Figure 2.6: RV32ICore MEM Stage

### 2.5.5 Write-Back (WB)

A Memory Extension unit sign- or zero-extends byte and halfword load results per the Funct3 field. A multiplexer selects among three write-back sources: the ALU

result, extended memory data (loads), or PC+4 (JAL/JALR return address). The selected value is written to rd on the rising clock edge.

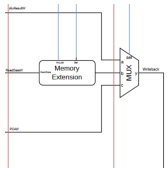



Figure 2.7: RV32ICore WB Stage

## 2.6 Hazard Unit

### 2.6.1 Data Hazards and Forwarding

When the destination register of an instruction in EX/MEM or MEM/WB matches a source register of the instruction in EX, the Hazard Unit selects the forwarded value at MuxX2 or MuxX3 instead of the register file output. When forwarding from MEM/WB after a load, the value is taken from the Memory Extension unit output.

### 2.6.2 Load-Use Hazards

When a load in EX is followed immediately by an instruction that consumes the loaded value, forwarding alone is insufficient. The Hazard Unit freezes the IF/ID and ID/EX registers for one cycle and inserts a NOP into EX, allowing the loaded value to reach MEM/WB where it can be forwarded.

### 2.6.3 Control Hazards

Branch and jump targets are resolved in EX. The two instructions fetched after the branch are discarded by flushing IF/ID and ID/EX in the same cycle the PC is updated to the target address.


Figure 2.8: RV32ICore Hazard Unit

## 2.7 Conclusion

This chapter covered the RV32I ISA, the architectural decisions behind RV32ICore's Harvard five-stage pipeline, and the mechanisms used to handle data, load-use, and

control hazards. These form the direct basis for the VHDL implementation in the following chapter.

# Chapter 3
# Implementation & Verification

## 3.1 Introduction

This chapter describes the VHDL implementation of RV32ICore and its simulation-based verification. Each architectural unit from Chapter 2 is realized as a distinct VHDL entity. Functional correctness is verified by compiling C programs of increasing complexity using the xPack RISC-V GCC toolchain, executing them in simulation, and comparing memory/register results against analytically computed expected values.

## 3.2 VHDL Design Organization

RV32ICore is organized as a hierarchy of VHDL entities. The top-level entity *P_Processor_Hazard* instantiates the datapath *(P_Datapath)*, control unit *(P_Control_Unit)*, and hazard unit *(Hazard_Unit)*. Memory is kept external to the processor entity, connected through top-level ports, allowing different memory configurations without modifying internal logic.

Top-level ports: clock, reset, 32-bit instruction input, 32-bit read data input, and outputs for PC, data address, write data, memory write enable, byte enable, and access mode.

## 3.3 Pipeline Register Implementation

All pipeline registers are implemented using a single generic VHDL component *RegEn*, parameterized by data width. It supports four modes: normal clocked operation, stall (output held), flush (NOP inserted), and synchronous reset.

The NOP value is `x"00000033"` (*ADD x0, x0, x0*) — a valid instruction that writes to x0, which is hardwired to zero, avoiding undefined behavior in decode/control logic during pipeline recovery. Narrower variants (*RegEn_5bit*, *RegEn_9bit*, *RegEn_6bit*) handle register addresses and reduced control buses.

## 3.4 Control Unit Implementation

*P_Control_Unit* contains three sub-components:

**P_Main_unit** — combinational decoder mapping opcode and Funct3 to all datapath control signals (immediate select, memory width, register/memory write enables, write-back select, PC select, ALU input select, branch mode).

**Alu_cntr** — decodes opcode, Funct3, and Funct7[5] to produce the 4-bit ALU operation code. Funct7[5] alone distinguishes arithmetic pairs (ADD/SUB, SRL/SRA).

**Excution_Flow** — resolves the final PC select in EX from the pipelined PC select, branch mode, ALU zero flag, and ALU LSB, producing a 2-bit output selecting PC+4, ALU result (JALR), or branch target.

### 3.4.1 Control Bus Pipelining

All control signals are packed into a single 21-bit bus at the Decode stage output and pipelined through *RegEn* registers in parallel with the datapath. At each stage boundary, only the relevant subset is extracted. The bus narrows progressively: 21-bit → 9-bit entering MEM (memory and write-back controls) → 6-bit entering WB (register write enable, memory extension, write-back select).

## 3.5 Datapath Implementation

### 3.5.1 Fetch Stage

The PC is a *RegEn* instance with stall input connected to *StallF*. *Pc_Next* is selected by *Mux3* among: ALU result (JALR), PC+4 (sequential), and branch target (taken branch). PC and PC+4 are registered into IF/ID for later stages.

### 3.5.2 Decode Stage

The register file is read combinationally using rs1[19:15] and rs2[24:20]. rd[11:7] is propagated through a dedicated 5-bit pipeline register chain to WB. The immediate extension unit takes the full instruction word and a 3-bit format select, producing a 32-bit sign-extended immediate. rs1 and rs2 are also registered separately as Rs1E and Rs2E for hazard unit comparisons.

### 3.5.3 Execute Stage

Forwarding multiplexers *ForwardingA* and *ForwardingB* (Mux3 instances) select among: register file output, WB result, or EX/MEM ALU result, controlled by

2-bit *ForwardA/ForwardB* from the Hazard Unit. The forwarded rs2 value is also registered into MEM via *ForwaredB_PLR4* for use by Store instructions. ALU input A is selected by *Mux3* (forwarded rs1, PC for AUIPC, or zero for LUI); ALU input B by *Mux2* (sign-extended immediate or forwarded rs2). The branch target is computed as the Decode-stage PC plus the sign-extended immediate.

### 3.5.4 Memory Stage

The data address is the EX/MEM ALU result. Write data is from *ForwaredB_PLR4*. The two LSBs of the ALU result are passed to the top level as *ByteEn* for sub-word memory access.

### 3.5.5 Write-Back Stage

*Mem_SignExt* sign- or zero-extends byte/halfword load results per Funct3 and the ALU result LSBs. The WB multiplexer selects among PC+4 (JAL/JALR), extended memory data (loads), or ALU result (all others), writing to the register file at the address in *Reg_A3_PLR5*.

## 3.6 Hazard Unit Implementation

*Hazard_Unit* is fully combinational, receiving source/destination register addresses from all pipeline stages, register write enables, write-back control LSB, and PC select MSB.

### 3.6.1 Forwarding Logic

MEM-stage forwarding takes priority over WB-stage forwarding (most recent value wins). Forwarding is suppressed when the destination is x0. Forwarding is also enabled when the PC select LSB is `'0'`, capturing JAL/JALR instructions that write PC+4 but are not flagged by the standard *RegWrite* path.

### 3.6.2 Load-Use Stall

Detected when the EX-stage instruction is a load ( *Writeback_cntr* LSB asserted) and its destination matches either source register in Decode. Response: assert *StallF* and *StallD* to freeze IF/ID and ID/EX, assert *FlushE* to insert a NOP into EX for one cycle.

### 3.6.3 Control Hazard Flush

*FlushE* = load-use stall OR PC select MSB. *FlushD* = PC select MSB alone. This discards the two incorrectly fetched instructions while leaving the Fetch stage free to begin fetching from the branch target.

## 3.7 Toolchain and Program Loading

Test programs are compiled using *riscv-none-elf-gcc* targeting RV32I (no compressed instructions or floating-point). A custom linker script places the text segment at `0x00000000` matching the reset vector, with the stack pointer initialized to the top of data memory. A startup assembly file sets the stack pointer and zeros the BSS section before transferring control to `main`.

```
int sum_array(int arr[], int len) {
        int sum = 0;
        for (int i = 0; i < len; i++) sum += arr[i];
        return sum;
}
int main() {
        int arr[] = {3, 7, 2, 9, 1};
        int s = sum_array(arr, 5); // expected: 22
        return 0;
}
```

```
00000000 <_start>:
0:  40000113   li   sp, 1024
4:  11400513   li   a0, 276
8:  11400593   li   a1, 276
0000000c <bss_loop>:
c:  00b55863   bge  a0, a1, 1c <bss_done>
10:  00052023   sw   zero, 0(a0)
14:  00450513   addi a0, a0, 4
18:  ff5ff06f   j    c <bss_loop>
0000001c <bss_done>:
1c:  07c000ef   jal  98 <main>
00000020 <halt>:
20:  0000006f   j    20 <halt>
```

**Note:** Full compiled output is provided in the Appendix.

The output ELF is converted to a hex file and loaded into simulation memory via:

```
mem load -infile output.hex -format hex
sim:/hazard_testbench/Inst_RAM/Mem
```

## 3.8 Simulation Environment

### 3.8.1 Testbench

```
1         Library ieee;
2         use ieee.std_logic_1164.all;
3         entity Hazard_Testbench is end entity;
4
5         architecture arch of Hazard_Testbench is
6         component P_Processor_Hazard is port(...); end component;
7         component RAM_Inst is port(...); end component;
8         component RAM is port(...); end component;
9         signal clk, reset : std_logic;
10        signal instruction, ReadData : std_logic_vector(31 downto 0);
11        signal MemWrite : std_logic;
12        signal ByteEn, Mode : std_logic_vector(1 downto 0);
13        signal Pc, DataAdr, WriteData : std_logic_vector(31 downto 0);
14        signal clk_cnt, Inst_Change_Cnt : integer := 0;
15        begin
16        UUT:      P_Processor_Hazard port map(...);
17        Data_RAM: RAM                port map(...);
18        Inst_RAM: RAM_Inst           port map(...);
19
20        clock: process begin
21        loop clk <= '1'; wait for 10 ns;
22        clk <= '0'; wait for 10 ns;
23        end loop;
24        end process;
25
26        Clock_counter: process(clk)
27        variable last : std_logic_vector(31 downto 0);
28        begin
29        if rising_edge(clk) then
30        clk_cnt <= clk_cnt + 1;
31        if instruction /= last then Inst_Change_Cnt <= Inst_Change_Cnt + 1; end
          ↪  if;
32        last := instruction;
33        end if;
34        end process;
35
36        Stimulus: process begin
37        reset <= '1'; wait for 40 ns;
38        reset <= '0'; wait;
39        end process;
40        end architecture;
```

The testbench applies reset for two clock cycles, initializing PC, register file, and

memory to zero. A 20 ns clock is generated via wait commands. *Inst_Change_Cnt* counts distinct instructions executed, used to measure performance.

## 3.9 Verification Tests

### 3.9.1 Test 1 — Sum Array

After reset, the PC initializes to 0 and increments by 4 each cycle. Figure 3.1 shows correct initialization behavior.

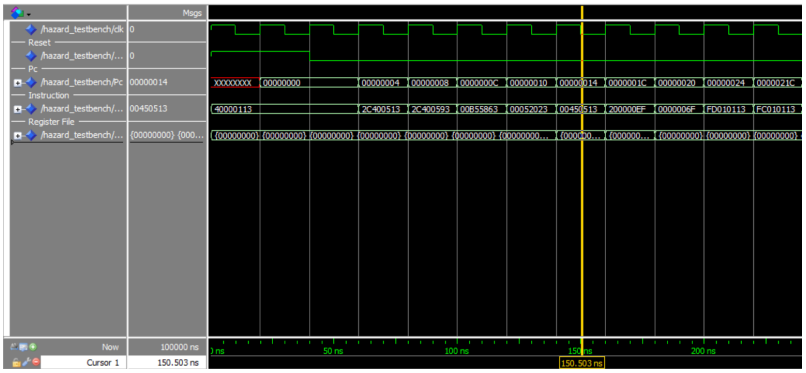



Figure 3.1: RV32ICore Initialization

At program completion, the result 0x16 (22) is stored at address `0x000003BC` (word index 0xEF in data memory), consistent with the expected sum of {3,7,2,9,1}.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 000000ff | 00000020 | 00000000 | 00000000 | 00000000 | 00000000 | 00000001 | 00000009 | 00000002 |
| 000000f7 | 00000007 | 00000003 | 00000000 | 00000000 | 000000E4 | 00000400 | 00000000 | 00000000 |
| 000000ef | 00000016 | 00000005 | 00000000 | 00000000 | 000003D8 | 00000005 | 00000000 | 00000000 |
| 000000e7 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 000000df | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 000000d7 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 000000cf | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 000000c7 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |

Figure 3.2: Sum Array Result

### 3.9.2 Test 2 — Multi-Function

A second test exercises sum_array, a multiplication-by-repeated-addition factorial, pointer-based find_max, and bitwise count_bits. Expected results: 0x16 (22), 0x78 (120), 0x09 (9), 0x08 (8).

```
1         int factorial(int n) {
2                 int result = 1;
3                 for (int i = 2; i <= n; i++) {
4                         int temp = 0;
5                         for (int j = 0; j < i; j++) temp += result;
```

```
6                       result = temp;
7               }
8               return result;
9       }
10      int find_max(int arr[], int len) {
11              int max = arr[0];
12              for (int i = 1; i < len; i++)
13              if (arr[i] > max) max = arr[i];
14              return max;
15      }
16      unsigned int count_bits(unsigned int n) {
17              unsigned int count = 0;
18              while (n) { count += n & 1; n >>= 1; }
19              return count;
20      }
21      int main() {
22              int arr[] = {3, 7, 2, 9, 1};
23              int s = sum_array(arr, 5);        // 22
24              int f = factorial(5);              // 120
25              int m = find_max(arr, 5);         // 9
26              unsigned int b = count_bits(255); // 8
27              return 0;
28      }
```

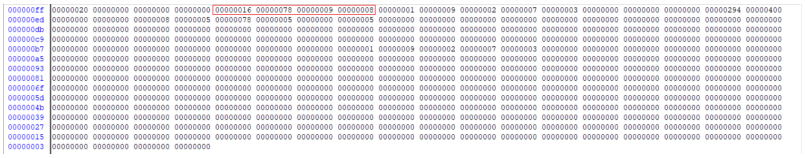


Figure 3.3: Second Test Results

All four results match expectations. Remaining RV32I instructions were verified through targeted test programs.

## 3.10 Conclusion

This chapter presented the complete VHDL implementation of RV32ICore and its simulation-based verification. Key implementation decisions — the generic *RegEn* component, progressive control bus narrowing, `ADD x0,x0,x0` as the NOP encoding, and the dedicated forwarded rs2 register for stores — reflect deliberate trade-offs between correctness, clarity, and reuse. Verification confirmed correct reset behavior, full RV32I instruction execution, and accurate results across both test programs, including loop execution, recursion, pointer manipulation, and bitwise operations.

# Chapter 4

# IDS Theoretical Background & System Architecture

## 4.1 Introduction

This chapter describes the architecture of the hardware IDS pipeline, covering its structural organization, protocol parsers, rule checkers, and detection logic.

## 4.2 Design Philosophy

The IDS is structured as a linear pipeline of independent modules sharing a common byte-stream interface: `rx_byte`, `rx_valid`, `rx_sof`, `rx_eof`. Parsers at different protocol layers operate simultaneously on the same data; rule checkers fire one clock cycle after their parser completes. Field extraction and rule evaluation are interleaved with frame reception — no full packet buffering is required.

The byte-stream interface makes each module independently testable and allows the physical reception mechanism (RMII simulation or AXI-Stream DMA) to be swapped without modifying any parser or checker.

## 4.3 Top-Level Structure

All sub-modules are wired together in `ids_top.vhd`. The byte stream is broadcast to every parser simultaneously.

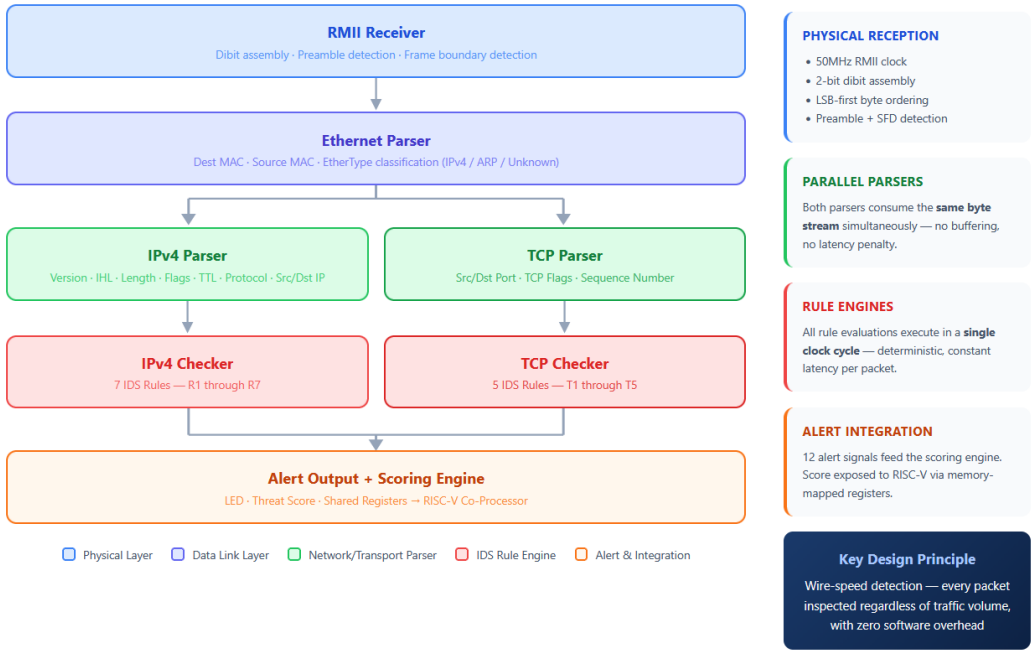



Figure 4.1: IDS pipeline architecture showing parallel parsers and rule engines.

Interface semantics: `rx_sof` pulses one cycle at frame start; `rx_byte`/`rx_valid` delivers one byte per clock; `rx_eof` pulses one cycle at frame end.

## 4.4 RMII Receiver (`rmii_rx.vhd`)

A three-state FSM (IDLE, PREAMBLE, DATA) assembles four consecutive dibits into one byte via a right-shift register. The preamble state counts at least eight `01` dibits before accepting the SFD (`11`). On hardware, this module is replaced by an AXI-Stream receiver without modifying the rest of the pipeline.

Listing 4.1: Dibit assembly in `rmii_rx.vhd`

```
shift_reg <= rmii_rxd & shift_reg(7 downto 2);
if dibit_cnt = "11" then
rx_byte  <= rmii_rxd & shift_reg(7 downto 2);
rx_valid <= '1';
end if;
```

## 4.5 Ethernet Parser (`eth_parser.vhd`)

Extracts destination MAC, source MAC, and EtherType from the first 14 bytes using a sequential FSM (DST_MAC, SRC_MAC, ETHERTYPE, PAYLOAD, DONE).

On DONE, asserts `frame_done` and one of `is_ipv4`, `is_arp`, or `is_unknown`. All outputs default to `'0'` and are overridden only in the DONE state, preventing latches.

## 4.6 Frame Counter (`frame_counter.vhd`)

Accumulates per-second statistics (total, IPv4, ARP, unknown frames) using a hardware divider. A `one_sec_tick` fires when the cycle counter reaches `CLK_FREQ_HZ - 1`, latching accumulators to output registers. `CLK_FREQ_HZ` is a generic: testbenches use 1000 (1 kHz tick after 1000 cycles); synthesis uses 50 000 000.

## 4.7 IPv4 Parser (`ipv4_parser.vhd`)

Skips the 14-byte Ethernet header then extracts the following IPv4 fields:

Table 4.1: IPv4 header fields extracted by `ipv4_parser.vhd`

| **Field** | **Byte offset** | **Width** |
|---|---|---|
| Version / IHL | 0 | 4+4 bits |
| Total Length | 2–3 | 16 bits |
| Flags / Fragment Offset | 6–7 | 3+13 bits |
| TTL | 8 | 8 bits |
| Protocol | 9 | 8 bits |
| Source IP | 12–15 | 32 bits |
| Destination IP | 16–19 | 32 bits |

The IHL field determines the header boundary (IHL×4 bytes), computed dynamically. `parse_done` and `parse_valid` (asserted only when `ip_version = 4`) are driven after the frame ends.

## 4.8 IPv4 Checker (`ipv4_checker.vhd`)

Receives parsed fields on `parse_done` and evaluates all rules combinatorially in one clock cycle. A local variable `any_alert` is OR'd across all conditions to drive `alert_any`; individual alert signals allow the scoring engine to weight rules differently.

## 4.9 TCP Parser (`tcp_parser.vhd`)

Captures `ip_ihl` and `ip_protocol` from the IPv4 parser output, then skips to byte $14 + \text{IHL} \times 4$ before extracting: source port, destination port, sequence number, and the flags byte (FIN, SYN, RST, PSH, ACK, URG at bits 0–5).

## 4.10 TCP Checker (`tcp_checker.vhd`)

Uses VHDL `alias` declarations for readable flag access:

Listing 4.2: Flag aliases in `tcp_checker.vhd`

```
alias flag_fin : std_logic is tcp_flags(0);
alias flag_syn : std_logic is tcp_flags(1);
alias flag_rst : std_logic is tcp_flags(2);
```

## 4.11 UDP Parser (`udp_parser.vhd`)

Reuses the TCP skip-to-transport logic, activated only when `ip_protocol = 0x11`. Extracts the 8-byte UDP header (source port, destination port, length, checksum).

## 4.12 UDP Checker (`udp_checker.vhd`)

Evaluates nine rules on extracted UDP fields against a configurable forbidden port set shared with the TCP checker through the rule register bank.

## 4.13 Detection Rules

### 4.13.1 IPv4 Rules (R1–R12)

Table 4.2: IPv4 detection rules R1–R7

| Rule | Condition | Attack / Anomaly |
|---|---|---|
| R1 | `ip_version` $\neq 4$ | Non-IPv4 / malformed |
| R2 | `ip_ihl` $< 5$ | Invalid header length |
| R3 | `ip_length` $< 20$ | Impossible packet size |
| R4 | `ip_ttl` $\leq 1$ | Suspicious TTL |
| R5 | `ip_frag_off` $> 0$ | Fragmentation attack |
| R6 | `ip_src` $=$ `0.0.0.0` | Spoofed source |
| R7 | `ip_src` $=$ `ip_dst` | Land attack |

Table 4.3: IPv4 detection rules R8–R12

| Rule | Condition | Attack / Anomaly |
|---|---|---|
| R8 | `ip_src[31:24]` = `0x7F` | Loopback source (127.x) |
| R9 | `ip_src[31:16]` = `0xA9FE` | Link-local (169.254.x) |
| R10 | `ip_src[31:28]` = `0xE` | Multicast source (224–239.x) |
| R11 | `ip_ihl` $> 5$ | IP options present |
| R12 | `ip_flags[2]` = `'1'` | Reserved flag bit set |

R8–R10 share a single `bogon_check_en` bit so the RISC-V can disable all three at once on networks where such addresses may legitimately appear.

### 4.13.2 TCP Rules (T1–T5)

Table 4.4: TCP detection rules T1–T5

| Rule | Condition | Attack / Anomaly |
|---|---|---|
| T1 | SYN=1 $\wedge$ FIN=1 | Illegal flag combination |
| T2 | SYN=1 $\wedge$ RST=1 | Illegal flag combination |
| T3 | All flags = `0x00` | NULL scan |
| T4 | All flags = `0xFF` | XMAS scan |
| T5 | `dst_port` $\in$ forbidden set | Forbidden service |

The forbidden port set (T5) defaults to Telnet (23), FTP (21), and TFTP (69) and is runtime-configurable by the RISC-V.

### 4.13.3 UDP Rules (U1–U9)

Table 4.5: UDP detection rules U1–U9

| Rule | Condition | Attack / Anomaly |
|---|---|---|
| U1 | `udp_dst_port` $\in$ forbidden set | Forbidden UDP service |
| U2 | `udp_length` $< 8$ | Invalid length |
| U3 | `udp_length` $= 0$ | Malformed UDP |
| U4 | `udp_src_port` $\in \{53,123,11211,19\}$ | Amplification response |
| U5 | `ip_src=ip_dst` $\wedge$ `src_port=dst_port` | UDP land attack |
| U6 | `udp_length` $> 1472$ | Oversized datagram |
| U7 | `src_port=0` $\vee$ `dst_port=0` | Reserved port zero |
| U8 | `udp_dst_port` $= 1900$ | SSDP/UPnP abuse |
| U9 | `udp_dst_port` $= 11211$ | Memcached amplification |

### 4.13.4 Alert Weighting

Table 4.6: Alert weights used by the scoring engine

| **Alert** | **Rule** | **Weight** |
|---|---|---|
| Bad version / IHL | R1, R2 | 5 |
| Bad length | R3 | 10 |
| Low TTL | R4 | 5 |
| Fragmentation | R5 | 15 |
| Spoofed source | R6 | 20 |
| Land attack | R7 | 25 |
| Bogon source | R8–R10 | 15 |
| IP options | R11 | 10 |
| Reserved flag | R12 | 10 |
| SYN+FIN / SYN+RST | T1, T2 | 25 / 20 |
| NULL / XMAS scan | T3, T4 | 20 |
| Forbidden TCP/UDP port | T5, U1 | 15 |
| Malformed UDP length | U2, U3 | 10 |
| Amplification resp. | U4 | 20 |
| UDP land attack | U5 | 25 |
| Oversized UDP | U6 | 10 |
| Port zero | U7 | 10 |
| SSDP/UPnP / Memcached | U8, U9 | 15 / 20 |

The score decays by a configurable rate every second when no alerts fire, preventing historical events from permanently elevating the severity level.

# Chapter 5
# Implementation & Testing

## 5.1 Methodology

Each checker module is verified independently by instantiating only the relevant parser–checker pair, injecting synthetic byte streams targeting boundary conditions, and checking alert signals in simulation. The full-pipeline testbench (`ids_top_tb.vhd`) then exercises the complete chain from RMII dibit injection to alert latch output.

A key optimization is the `CLK_FREQ_HZ` generic override:

Listing 5.1: Generic override in the testbench

```
u_dut : entity work.ids_top
generic map (CLK_FREQ_HZ => 1000)
port map (...);
```

At 1 kHz, the frame counter's one-second tick fires after 1000 cycles instead of 50 000 000, making counter behaviour observable without long simulation runs.

## 5.2 Test Procedures

VHDL procedures encapsulate frame injection, driving RMII signals cycle-by-cycle including preamble, SFD, header bytes, and EOF. A complete test case reduces to a few lines:

Listing 5.2: Injecting a land attack

```
send_ipv4_frame(
src_ip   => x"C0A80101",
dst_ip   => x"C0A80101",
protocol => x"06",
ttl      => x"40"
);
wait until rising_edge(clk);
```

```
assert alert_land_attack = '1'
report "Land attack not detected" severity failure;
```

## 5.3 IPv4 Checker Verification

All twelve R1–R12 rules were verified in a single simulation run. Each frame triggers exactly one rule; no secondary alerts were observed.

**Header Validation (R1–R7):** R1: `ip_version=6` → `alert_bad_version`; R2: `ip_ihl=3` → `alert_bad_ihl`; R3: `ip_length=10` → `alert_bad_length`; R4: `ip_ttl=1` → `alert_low_ttl`; R5: non-zero fragment offset → `alert_fragment`; R6: `ip_src=0.0.0.0` → `alert_spoof_src`; R7: `ip_src=ip_dst` → `alert_land_attack`.

**Bogon Address Rules (R8–R10):** R8: `ip_src=127.0.0.1` → `alert_bogon_loop`; R9: `ip_src=169.254.0.1` → `alert_bogon_link`; R10: `ip_src=224.0.0.1` → `alert_bogon_mcast`

**Header Abuse Rules (R11–R12):** R11: `ip_ihl=6` → `alert_ip_options`; R12: `ip_flags[2]='1'` → `alert_reserved_bit`.

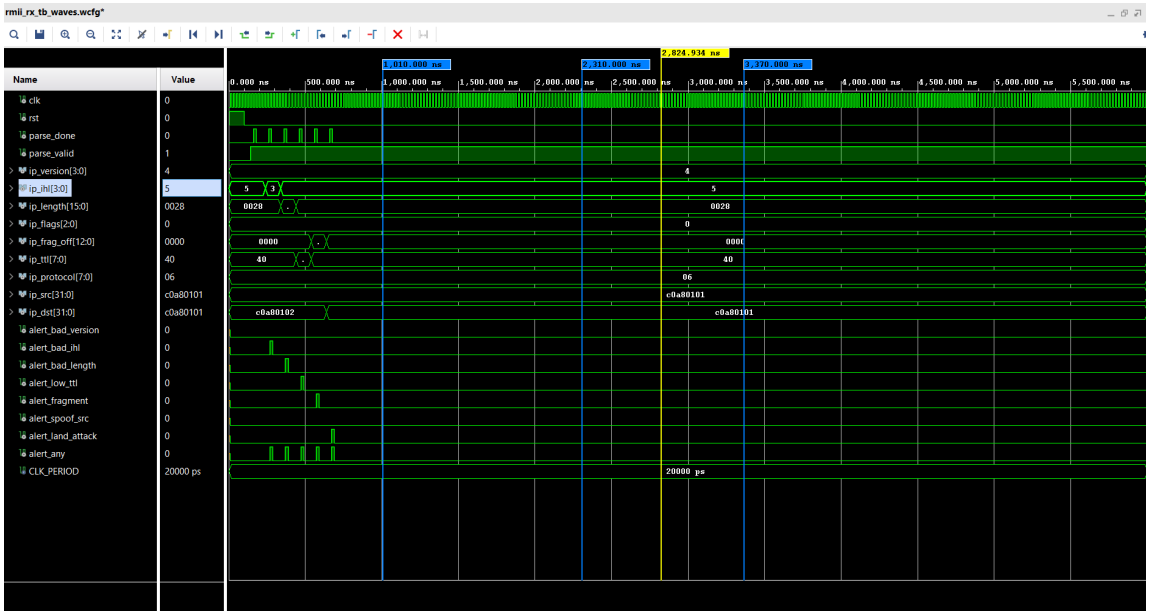



Figure 5.1: Simulation waveform showing R1-R7 IPv4 IDS rules firing sequentially.


Figure 5.2: Simulation waveform showing R8-R12 IPv4 IDS rules firing sequentially.

## 5.4 TCP Checker Verification

T1: `tcp_flags=0x03` (SYN+FIN) → `alert_syn_fin`; T2: `tcp_flags=0x06` (SYN+RST) → `alert_syn_rst`; T3: `tcp_flags=0x00` (NULL) → `alert_null_scan`; T4: `tcp_flags=0xFF` (all flags) → `alert_xmas_scan`; T5: `dst_port=23` (Telnet) → `alert_forbidden`.


Figure 5.3: Simulation waveform showing TCP rules T1–T5 firing in sequence.

## 5.5 UDP Checker Verification

All nine U1–U9 rules were verified. Each alert fired within the expected number of clock cycles from header assembly; no secondary alerts were triggered on single-violation frames.

**Forbidden Port / Structural (U1–U3):** U1: `udp_dst_port=69` → `alert_udp_forbidden`; U2: `udp_length=4` → `alert_udp_short`; U3: `udp_length=0` → `alert_udp_zero`.

**Amplification / Spoofing (U4–U5):** U4: `udp_src_port=53` → `alert_udp_amplify`; U5: `ip_src=ip_dst` ∧ `src_port=dst_port` → `alert_udp_land` (confirmed that satisfying only one condition produces no alert).

**Size / Service Abuse (U6–U9):** U6: `udp_length=1480` → `alert_udp_oversize`; U7: `udp_dst_port=0` → `alert_udp_port_zero`; U8: `udp_dst_port=1900` → `alert_ssdp_abuse`; U9: `udp_dst_port=11211` → `alert_memcached`.

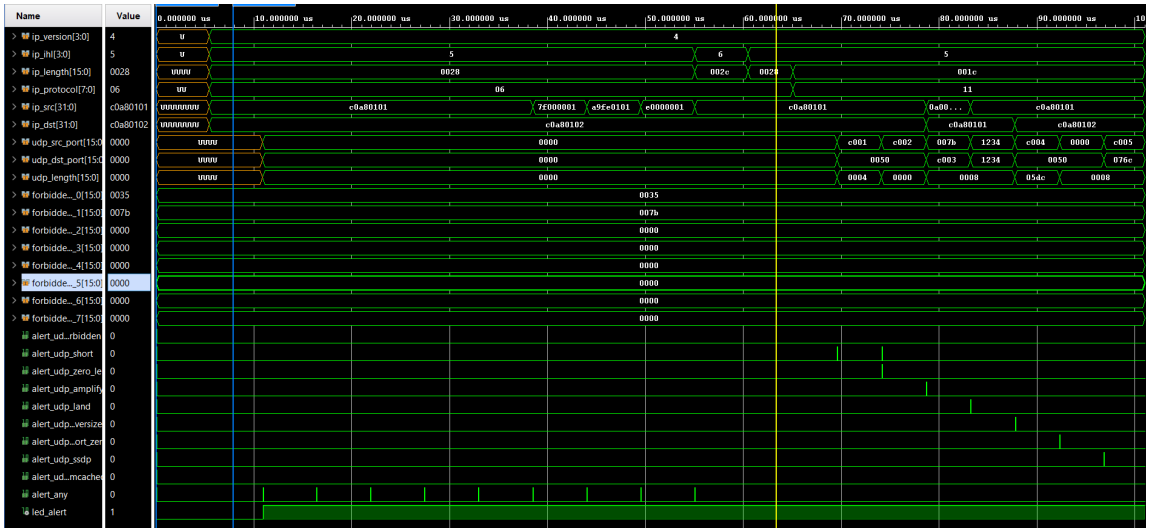



Figure 5.4: Simulation waveform showing all UDP IDS rules firing sequentially.

# Chapter 6
# System Integration

## 6.1 Hardware Platform

The MicroPhase Z7-Lite combines an ARM Cortex-A9 PS and Artix-7-equivalent PL in a single Zynq-7020 SoC. The on-board RTL8201F Ethernet PHY connects to the PS via RGMII, so the MAC and PHY interface are handled entirely within the PS. The PL cannot directly observe raw Ethernet traffic; raw packet bytes must be passed from PS to PL.

Table 6.1: Z7-Lite PL pin assignments

| **Signal** | **Pin** | **Function** |
|---|---|---|
| `PL_CLK_50M` | N18 | 50 MHz main clock |
| `PL_LED1` | P15 | `led_alert` output |
| `GPIO1_17N` | U12 | `led_active` output |
| `PL_KEY1` | P16 | Active-low reset |

## 6.2 PS-to-PL Packet Forwarding

The PS ARM receives complete Ethernet frames via the GigE MAC, strips the 4-byte FCS, and writes the frame into a DMA buffer. The AXI-Stream DMA IP transfers the buffer to PL fabric, where `axis_rx.vhd` converts AXI-Stream signals (`tvalid`, `tready`, `tdata`, `tlast`) into the internal byte-stream interface. The entire parser and checker stack is unchanged for hardware use.


Figure 6.1: PS-PL co-design: packet path from RTL8201F through AXI-Stream DMA to the IDS pipeline, with shared register bank connecting IDS to RV32ICore.

The `axis_rx.vhd` wrapper asserts `tready` permanently and maps: `tvalid`→`rx_valid`, `tdata[7:0]`→`rx_byte`, `tlast`→`rx_eof`, and a one-cycle delayed `tvalid` rising edge→`rx_sof`.

## 6.3 RISC-V Co-Processor Integration

### 6.3.1 Shared Register Bank

RV32ICore exposes a standard external memory interface (`DataAdr`, `ReadData`, `WriteData`, `MemWrite`, `ByteEn`, `Mode`), so IDS registers appear to the processor as ordinary memory locations. The shared register bank (`shared_regs.vhd`) implements 29 32-bit registers at addresses `0x400–0x4B4`:

- **Status registers (0x400–0x454)** — written by IDS, read by RISC-V: threat score, 12-bit alert flags bitmask, cumulative alert counts, per-protocol frame counts, frames-per-second statistics, and last-alerted packet fields.
- **Rule registers (0x480–0x4B4)** — written by RISC-V, read by IDS: TTL threshold, fragmentation check enable, scoring thresholds, score decay rate, alert clear command, and eight configurable forbidden port values.

Table 6.2: Shared register map (selected entries)

| Address | Register | Direction |
|---|---|---|
| 0x400 | `threat_score` | IDS → RISC-V |
| 0x404 | `alert_flags` | IDS → RISC-V |
| 0x428 | `fps_total` | IDS → RISC-V |
| 0x434 | `last_alert_src_ip` | IDS → RISC-V |
| 0x454 | `threat_level` | IDS → RISC-V |
| 0x480 | `ttl_threshold` | RISC-V → IDS |
| 0x490 | `score_decay_rate` | RISC-V → IDS |
| 0x494 | `alert_clear` | RISC-V → IDS |
| 0x498 | `forbidden_port_0` | RISC-V → IDS |
| 0x800 | `uart_tx` | RISC-V → UART |

### 6.3.2 Adaptive Threat Scoring

The threat score accumulates alert weights each clock cycle and decays by a configurable rate every second. The RISC-V polls the score register at one-second intervals and maps it to three severity levels:

- **GREEN** (score $< 30$): Normal traffic; default rule thresholds.
- **YELLOW** ($30 \leq$ score $< 100$): Elevated activity; RISC-V tightens TTL threshold and lowers fragmentation sensitivity.
- **RED** (score $\geq 100$): Active attack; RISC-V sets maximum TTL threshold and enables all optional checks.

### 6.3.3 UART Alert Output

The RISC-V formats alert data (severity level, triggered rule, source IP) as human-readable strings and transmits them over UART through a memory-mapped transmitter at address `0x800`.

## 6.4 Packet Capture Module (`packet_capture.vhd`)

On each `frame_done` pulse, all parsed header fields (Ethernet MACs, EtherType, IPv4 fields, ports, TCP flags, sequence number) are latched into RISC-V-readable registers. A `capture_mode` bit selects between logging every frame or only alert-triggering frames.

## 6.5 Future Hardware Integration

The simulation-verified pipeline is ready for deployment. Remaining steps:

1. Create a Vivado block design for xc7z020clg400-1 with Zynq PS configured for GigE MAC with DMA, AXI-Stream DMA IP, and the IDS PL design as a custom IP block.
2. Write a minimal PS bare-metal C driver in Vitis to forward frames from the GigE MAC to PL via DMA.
3. Apply XDC constraints from Table 6.1.
4. Validate with Scapy-generated test packets over Ethernet.

If hardware integration is not completed before the defense, the simulation results constitute full validation of IDS functionality.

# Chapter 7
# Conclusion

## 7.1 Summary of Achievements

This project produced a fully simulation-verified hardware IDS pipeline comprising eight VHDL modules and twenty detection rules across the Ethernet, IPv4, TCP, and UDP protocol layers. Every rule was verified individually and validated again in a full end-to-end testbench exercising the complete chain from raw dibit input to alert latch output.

The integration architecture connects the IDS to RV32ICore through a 29-register memory-mapped bank, giving the processor read access to live traffic statistics and write access to configurable rule parameters without modifying the IDS hardware. The result combines deterministic hardware inspection throughput with software-processor adaptability.

## 7.2 Academic Contribution

This project extends typical hardware IDS work in two respects:

1. **Runtime reconfigurability:** Detection thresholds and forbidden port lists can be updated through the RISC-V without FPGA reprogramming.
2. **Adaptive severity:** The weighted scoring engine with decay creates a feedback loop between traffic behaviour and rule sensitivity, approximating an adaptive IDS in hardware.

## 7.3 Limitations

The system does not inspect packet payloads; all rules operate on header fields only, excluding application-layer attacks (SQL injection, XSS, etc.). Hardware integration with the Zynq PS has been designed and planned but not yet validated on physical hardware; simulation results are the primary evidence of correctness.

## 7.4 Future Work

Planned extensions include: a `payload_extractor.vhd` module buffering TCP/UDP payload into dual-port BRAM for RISC-V signature search; a configurable BRAM-based IP blacklist; ICMP parsing for reconnaissance detection (ping sweeps, redirect attacks); and hardware bring-up on the Z7-Lite for wire-speed validation on a live network segment.

# Bibliography


[1] J. Postel, *Internet Protocol*, IETF RFC 791, September 1981.

[2] J. Postel, *Transmission Control Protocol*, IETF RFC 793, September 1981.

[3] J. Postel, *User Datagram Protocol*, IETF RFC 768, August 1980.

[4] IEEE Std 802.3-2022, *IEEE Standard for Ethernet*, IEEE, 2022.

[5] Xilinx Inc., *Zynq-7000 SoC Technical Reference Manual*, UG585, v1.13, 2022.

[6] Xilinx Inc., *AXI DMA LogiCORE IP Product Guide*, PG021, v7.1, 2021.

[7] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V Foundation, 2019.

[8] G. Lyon, *Nmap Network Scanning*, Insecure.com LLC, 2009.

[9] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. USENIX LISA*, 1999.

[10] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2021.

[11] S. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.

[12] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[13] P. J. Ashenden, *The Designer's Guide to VHDL*, 3rd ed. Morgan Kaufmann, 2008.

[14] V. A. Pedroni, *Circuit Design and Simulation with VHDL*, 2nd ed. MIT Press, 2010.

[15] AMD Xilinx, *Vivado Design Suite User Guide: Design Flows Overview*, UG892, 2023.

[16] AMD Xilinx, *7 Series FPGAs Data Sheet: Overview*, DS180, 2023.

[17] xPack Project, *xPack GNU RISC-V Embedded GCC*. [Online]. Available: `https://xpack.github.io/riscv-none-elf-gcc/`

[18] Free Software Foundation, *Using ld: The GNU Linker*. [Online]. Available: `https://sourceware.org/binutils/docs/ld/`

[19] Free Software Foundation, *Using as: The GNU Assembler*. [Online]. Available: `https://sourceware.org/binutils/docs/as/`

[20] Siemens EDA, *ModelSim User's Manual*. Mentor Graphics, 2023.